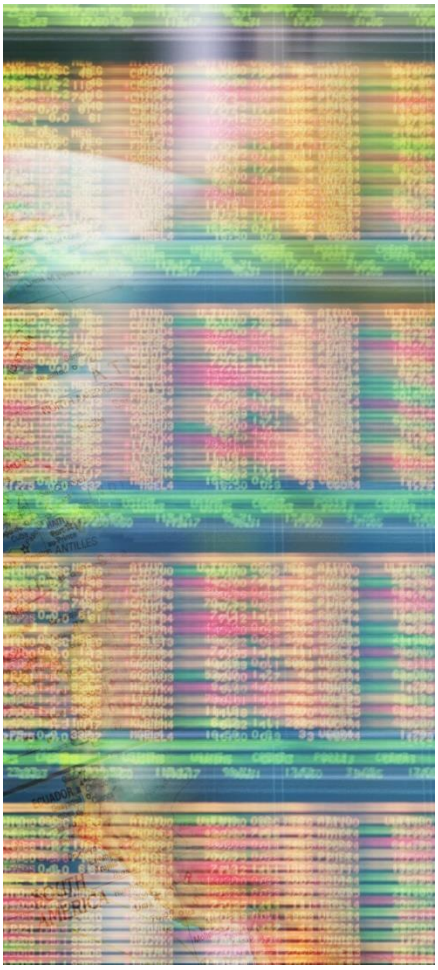
# Guía de optimización

Los Procesadores Escalables de Intel® Xeon®

# La Guía para optimizar Intel® Select Solutions for Genomics Analytics en la Plataforma de Procesadores Escalables Intel® Xeon® de 3a Generación



## Índice de contenidos


**1. Introducción Intel® Select Solutions for Genomics Analytics ..... 3**

1.1. La arquitectura ..... 4

**2. Optimización del Intel® Select Solutions for Genomics Analytics ..... 5**

2.1. Optimizar el clúster de computación de alto rendimiento (HPC) ..... 5

2.2. Configurar el Gestionador de Carga de Trabajo Slurm ..... 5

2.2.1. Instalar el servidor Slurm en el primer nodo ..... 5

2.2.2. Actualizar la información de los recursos del nodo informático ..... 6

2.2.3. Instalar el cliente Slurm en una imagen del nodo informático: ..... 9

2.2.4. Opcional – Instalar el cliente Slurm en el primer nodo ..... 10

2.2.5. Completar la configuración de Slurm ..... 11

2.2.6. Verificar la configuración de Slurm ..... 12

2.3. Instalar las herramientas de análisis genómico ..... 13

2.3.1. Verificar la configuración del entorno del clúster genómico Primero, verifique que la configuración del clúster que usa para ejecutar el análisis genómico sea correcta. ..... 13

2.3.2. Configurar la base de datos, MariaDB ..... 13

2.3.3. Instalar el sistema de gestión de flujo de trabajo del Cromwell ..... 14

2.3.4. Configurar el entorno Cromwell ..... 17

2.3.5. Verificar la configuración de Cromwell ..... 21

2.3.6. Instalar el paquete software Burrows-Wheeler Aligner (BWA) ..... 23

2.3.7. Instalar el kit de herramientas de análisis genómico (GATK) ..... 24

2.3.8. Instalar Picard ..... 25

2.3.9. Instalar Samtools ..... 25

2.3.10. Instalar VerifyBamID2 ..... 26

**3. Verificar la configuración ..... 27**

**4. Resolviendo problemas con el software genómicas ..... 28**

4.1. Resolviendo problemas iniciando una sesión con MariaDB ..... 28

**5. Instalar componentes opcionales ..... 28**

**6. Conclusión ..... 28**

| Fecha | Rev | Descripción |
|---|---|---|
| 12-21-2021 | 1.0 | La primera versión |

La Guía para optimizar Intel® Select Solutions for Genomics Analytics en la Plataforma de Procesadores Escalables Intel® Xeon® de 3a Generación

# 1. Introducción Intel® Select Solutions for Genomics Analytics

Esta guía es para usuarios del Intel® Select Solutions for Genomics Analytics. Incluye las recomendaciones para configurar el BIOS, el sistema operativo (OS) y el software de análisis genómico con los ajustes que se puede aumentar el rendimiento en muchas situaciones. En la guía llamada, *HPC Cluster Tuning on 3rd Generation Intel® Xeon® Scalable Processors*, le recomienda que aumente el rendimiento usando las configuraciones del hardware. En esta guía, le recomendamos que aumente el rendimiento usando estas configuraciones del software. Tenemos en cuenta que confiamos en que los usuarios consideren cuidadosamente todas las configuraciones porque los escenarios específicos del Intel® Select Solutions for Genomics Analytics se pueden implementar de varias maneras.

El análisis genómico se logra usando estas aplicaciones del software:

- El kit de herramientas de análisis genómico (GATK por su sigla en inglés) Versión 4.1.9.0 es usando para analizar datos de secuenciación genética y para el descubrimiento de variantes. Es la norma para identificación de SNPs e "indels" en el ADN de la línea germinal y datos ARNseq. Incluye utilidades para hacer tareas como procesar y controlar la calidad de datos de secuencia de alto rendimiento.

- La versión Cromwell 52 es un sistema de gestión de flujo de trabajo adecuado para trabajos científicos. Es un programador que se puede utilizar cuando encadenamos procesos. Cromwell utiliza el formato de idioma que es el Lenguaje de Descripción del Flujo de Trabajo (WDL por su sigla en inglés) para describir los flujos de trabajo. Cromwell usa el MariaDB para realizar un seguimiento de los flujos de trabajo y archivar los resultados de las tareas.

- La versión Burrows-Wheeler Aligner (BWA) 0.7.17 se usa para comparar las secuencias del ADN con un genoma de referencia como el genoma de referencia humano.

- La versión Picard 2.21.1 es un conjunto de aplicaciones que se usa para traducir los datos de la secuencia de alto rendimiento entre formas de archivos como: SAM, BAM, CRAM o VCF.

- La versión VerifyBAMID2 con Samtools 1.9 se usa para comparar un fragmento del ADN con genotipos previamente conocidos. También se puede usar para encontrar contaminación en un fragmento del ADN.

- La versión Samtools 1.9 se usa para manipular los datos de la secuencia de alto rendimiento incluyendo para: leer, escribir, editar, mirar o poner un índice a los datos que han guardado en las formas de SAM, BAM, o CRAM. Se usa para leer o para escribir las formas BCF2, VCF o gVCF. También, se usa para llamar, para filtrar o para resumir SNP y variantes de secuencia genética corta.

- Para hacer una evaluación comparativa, usa la Prueba de Alto Rendimiento que Ejecuta 20.000 Veces (en el inglés: 20K Throughput Run). Esta prueba es usada para verificar que su configuración del clúster de la computación de alto rendimiento (en inglés: HPC cluster) pueda ejecutar correctamente en un genoma de referencia o en una secuenciación de exoma completo. Se usa para verificar que las funcionas más importantes pueden funcionar bien.

- Opcional: la versión Intel® System Configuration Utility (SYSCFG) 14.2 Build 8 es una utilidad de línea de comandos para cargar y recargar el sistema BIOS y para gestionar los ajustes del firmware.

Los prerrequisitos del software incluyen:

- Git se usa para el control de las versiones del software

- Java 8 o Java Runtime Environment (JRE) y Software Developers Kit (SDK) 1.8 son necesarios para ejecutar el Cromwell.

- La versión Python 2.6 o más recientes, es necesario para usar el gatk-launch. Otros paquetes del software usan la versión Python 3.6.2 o más recientes. También se usan otros paquetes del Python. Se usa Conda para gestionar el entorno y para ir aislando las dependencias del software.

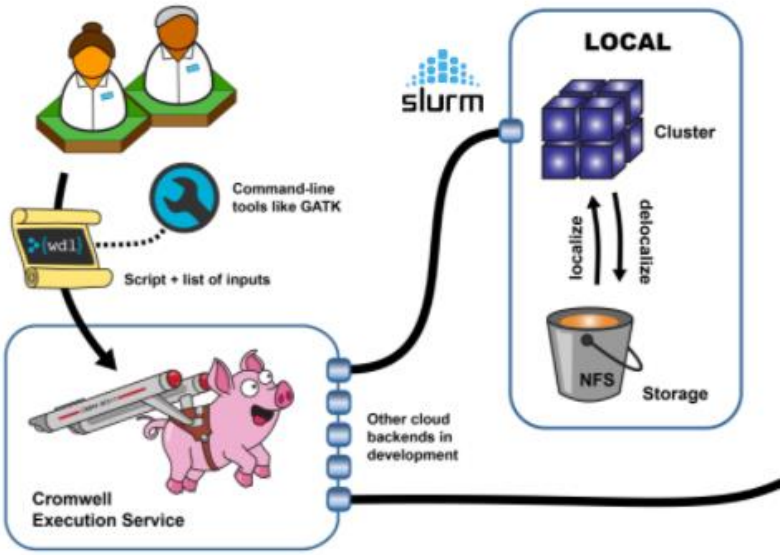
- Gestionador de la Carga de Trabajo Slurm se usa para crear una agenda de trabajos. También, se puede manejar los recursos del clúster del HPC. Puede obtener más información: https://slurm.schedmd.com/documentation.html.
- sbt es necesario para compilar el Cromwell
- Cromwell usa MariaDB para guardar los datos de almacenamiento
- GATK usa R, Rscript, gsalib, ggplot2, reshape, and gplots para producir gráficos y diagramas

Los procesadores escalables Intel® Xeon® de 3a Generación contienen plataformas que permiten la optimización de las cargas de trabajo con aceleración de la IA (Inteligencia Artificial) incorporada. Estos procesadores tienen el rendimiento que ayudan acelerar el impacto transformador de los datos desde el perímetro (Edge) a la nube (Cloud). Los mejoramientos específicos del análisis genómico incluyen:

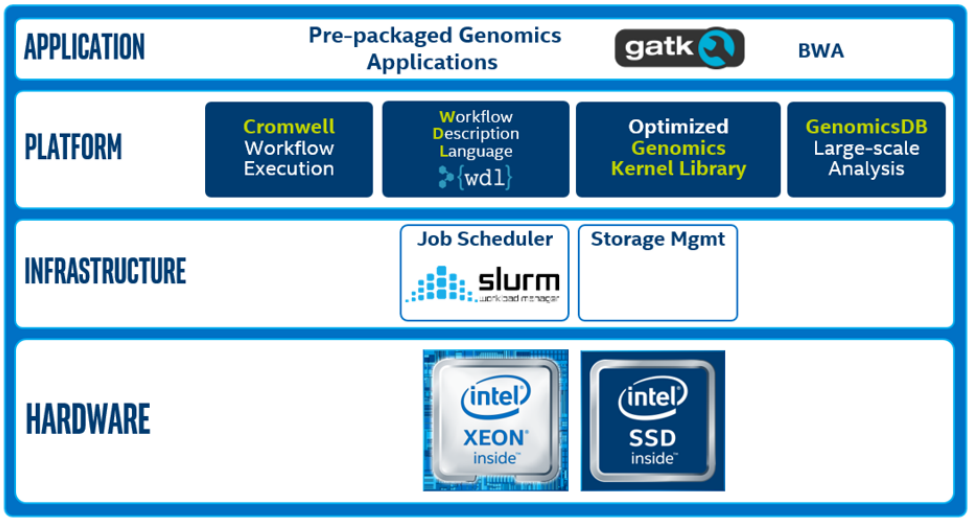
- El rendimiento mejorado
- Más Intel® Ultra Path Interconnect
- Intel® Advanced Vector Extensions
- Intel® Genomics Kernel Library con AVX-512

## 1.1. La arquitectura

El diagrama de abajo muestra el flujo de los datos por Intel® Select Solutions for Genomics Analytics. Los científicos usan aplicaciones como el kit de herramientas de análisis genómico y los códigos WDL para procesar los datos ADN. El Cromwell se usa para manejar el flujo de trabajo. El Slurm se usa para crear una agenda de trabajo y para manejar los recursos del clúster del HPC.



El diagrama de abajo muestra el software y el hardware que se usa en Intel® Select Solutions for Genomics Analytics.




## 2. Optimización del Intel® Select Solutions for Genomics Analytics

La optimización de la configuración del software es necesario. La configuración base fue diseñado para aplicaciones generales. Tiene que optimizar su software para lograr un mejor rendimiento del sistema. Las secciones siguientes tienen instrucciones paso a paso para ajustar el Intel® Select Solutions for Genomics Analytics.

### 2.1. Optimizar el clúster de computación de alto rendimiento (HPC)

Las recomendaciones que se usa para optimizar el hardware se encuentran en esta guía: *HPC Cluster Tuning on 3rd Generation Intel® Xeon® Scalable Processors,* https://www.intel.com/content/www/us/en/developer/articles/guide/hpc-cluster-tuning-on-3rd-generation-xeon.html

### 2.2. Configurar el Gestionador de Carga de Trabajo Slurm

Cuando se agrega el Slurm al HPC clúster, se instala el servidor de Slurm en el primer nodo.

#### 2.2.1. Instalar el servidor Slurm en el primer nodo

El Slurm necesita un usuario del sistema para ejecutar los procesos que manejan los recursos. La configuración predeterminada de OpenHPC requiere un usuario "slurm". PAM (Pluggable Authentication Module) prohíbe a un usuario normal usar SSH para obtener el acceso a los nodos a través de Slurm. La biblioteca Munge de autenticación tiene el archivo con la configuración Slurm global y la clave cifrada instalan en todos los nodos.

El archivo de la configuración global de Slurm y la clave cifrada que son requeridas por la biblioteca de autenticación Munge deben estar disponibles en todos los nodos en el grupo que se usa para controlar los recursos (en inglés: resource management pool) .

Crear un grupo de usuarios que obtenga el acceso SSH sin restricciones. Use estas instrucciones para agregar nuevos usuarios a este grupo:

1. Crear un grupo de usuarios que obtenga el acceso SSH sin restricciones:
   ```
   groupadd sshallowlist
   ```

2. Agregue a la lista blanca al usuario que es de Intel® Cluster Checker. Este usuario debe poder ejecutar el Cluster Checker a dentro y afuera un trabajo que se usa controlar recursos.
   `usermod -aG sshallowlist clck`
3. Cree la cuenta de usuario Slurm:
   `useradd --system --shell=/sbin/nologin slurm`
4. Instale los paquetes del servidor Slurm:
   `dnf -y install ohpc-slurm-server`
5. Actualice los archivos Warewulf:
   `wwsh file sync`

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
groupadd sshallowlist

usermod -aG sshallowlist clck

useradd --system --shell=/sbin/nologin slurm

dnf -y install ohpc-slurm-server

wwsh file sync
```

### 2.2.2. Actualizar la información de los recursos del nodo informático

Actualice el archivo que contiene la configuración Slurm. Agregue los nombres de los nodos informáticos, las propiedades de sus procesadores y las particiones o colas que se usa con su clúster HPC. Agregue las siguientes especificaciones:

- NodeName tiene que contener los nombres de los nodos informáticos con las definiciones de sus capacidades.
- Sockets contiene el número de sockets en un nodo informático.
- CoresPerSocket contiene el número de núcleos de CPU en cada socket.
- ThreadsPerCore contiene el número de procesos que se pueda ejecutar en paralelo en un núcleo.
- PartitionName, contiene la definición de las particiones Slurm, o colas, donde los nombres de los nodos informáticos son asignados. Usuarios pueden usar estas especificaciones para acceder a los recursos. El usuario puede ver el nombre de la partición.
- Nodes contiene una lista de todos los nodos informáticos que pertenecen a una partición.
- Default contiene la partición predeterminada. Si un usuario no especifica explícitamente cual partición usar, entonces la partición predeterminada se usará.

Haga estas tareas para crear el archivo de configuración Slurm de su clúster que es basado en la lista de materiales de este diseño de referencia:

- Use la plantilla openHPC para crear un archivo de configuración Slurm nuevo.
- Agregue el ajuste, ControlMachine. Este ajuste tiene que contener el nombre del primer servidor.
- Cambie su archivo de la configuración Slurm para que el proceso que controla Slurm hará un nodo disponible si este nodo deje de funcionar.

- Actualice las capacidades del hardware en la especificación, NodeName.
- Actualice la definición de las particiones Slurm en la especificación, PartitionName.

**Instrucciones paso a paso**

1. Para crear el archivo de configuración Slurm nuevo, copie la plantilla openHPC Slurm:
   ```
   cp /etc/slurm/slurm.conf.ohpc /etc/slurm/slurm.conf
   ```

2. Abre el archivo, /etc/slurm/slurm.conf. Haga estos cambios:

   a. Encuentra la linea que se inicia con la palabra "ControlMachine" y actualícelo con:
   ```
   ControlMachine=frontend
   ```

   b. El archivo de la configuración Slurm en el paquete OpenHPC contiene una configuración donde el proceso que controla Slurm hará un nodo disponible después de que deje de funcionar. La razón de que deje de funcionar es porque este nodo no responde. Si hay otra razón entonces el proceso que controla Slurm no reiniciará este nodo.
   Esta configuración predeterminada se puede usar cuando un clúster grande pueda ser supervisado constantemente por un administrador de sistema. Puede obtener más información en: https://slurm.schedmd.com/slurm.conf.html.

   Si tiene un clúster más pequeño o un clúster que no se pueda ser supervisado constantemente, es mejor cambiar la configuración. El proceso que controla Slurm debe reiniciar el nodo cuando un nodo tiene una configuración válida y también este nodo está registrado con el proceso que controla Slurm. Para cambiar la configuración para permitir este proceso, encuentre en esta línea:

   ```
   ReturnToService=1
   ```

   Actualícelo con:
   ```
   ReturnToService=2
   ```

   c. El archivo de configuración Slurm en el paquete OpenHPC permite que los trabajos puedan compartir un nodo informático. Ajuste la configuración del horario para que Slurm pueda considerar dos cosas: cuantos CPUs se necesita para hacer una tarea y cuanta memoria se necesita para hacer una tarea.

   Encuentre la línea:
   ```
   SelectType=select/cons_tres
   ```

   Actualícelo con:
   ```
   SelectType=select/cons_res
   ```

   *Nota: Ten cuidado eliminar el "t" y usa el ajuste: select/cons_res*

   Entonces, encuentre la línea:
   ```
   SelectTypeParameters=CR_Core
   ```

   Actualícelo con:
   ```
   SelectTypeParameters=CR_Core_Memory
   ```

   d. La información nodo es basado en la configuración del clúster, por ejemplo, la Plataforma de Procesadores Escalables Intel® Xeon® de 3a Generación Oro 6348 (en inglés: 3rd Generation Intel® Xeon® Scalable Gold 6348). Encuentra la línea que contiene la especificación, NodeName. Actualícelo con:

   ```
   NodeName=c[01-04] Sockets=2 CoresPerSocket=28 ThreadsPerCore=2RealMemory=480000
   state=UNKNOWN
   ```

   bwa, all y haplo se pueda usar para mejorar el rendimiento cuando está usando Cromwell. Encuentre

las líneas que contienen las especificaciones, PartitionName. Actualícelo con estas líneas:

```
PartitionName=xeon Nodes=c[01-04] Priority=10000 default=YES MaxTime=24:00:00 State=UP
PartitionName=bwa Nodes=c[01-04] Priority=4000 Default=NO MaxTime=24:00:00 State=UP
PartitionName=all Nodes=c[01-04] Priority=6000 Default=NO MaxTime=24:00:00 State=UP
PartitionName=haplo Nodes=c[01-04] Priority=5000 default=NO MaxTime=24:00:00 State=UP
```

e. Hay un error tipográfico en el archivo de configuración Slurm. Porque hay un error, el paquete OpenHPC no permite iniciar slurmctld y slurmd, tiene que cambiar esta línea por una acotación.

Encuentra la línea:
`JobCompType=jobcomp/none`

Actualícelo. Agregue el símbolo "#" al inicio:
`#JobCompType=jobcomp/none`

f. Guarde y cierre el archivo.

3. Importe los archivos configuraciones nuevos a Warewulf:

```
wwsh -y file import /etc/slurm/slurm.conf
```

4. Actualice el archivo: */etc/warewulf/defaults/provision.conf*

a. Abre el archivo: /etc/warewulf/defaults/provision.conf file. Encuentra la línea que se inicia con esta frase:
`files = dynamic_hosts, passwd, group ...`

*Importante: No cambie nada en esta linea. Agregue el siguente texto al final. Recuerda la coma al principio.*

```
, slurm.conf, munge.key
```

b. Guarde y cierre el archivo.

5. Permite los servicios de control de Munge y Slurm en el primer nodo:

```
syssystemctl enable slurmctld.service
```

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
cp /etc/slurm/slurm.conf.ohpc /etc/slurm/slurm.conf


sed -i "s/^\(NodeName.*\)/#\1/" /etc/slurm/slurm.conf


echo "NodeName=${compute_prefix}[${first_node}-${last_node}] Sockets=${num_sockets} \
CoresPerSocket=${num_cores} ThreadsPerCore=2 State=UNKNOWN" >> /etc/slurm/slurm.conf
sed -i "s/ControlMachine=.*/ControlMachine=${frontend_name}/" /etc/slurm/slurm.conf
sed -i "s/^\(PartitionName.*\)/#\1/" /etc/slurm/slurm.conf
```

```
sed -i "s/^\(ReturnToService.*\)/#\1\nReturnToService=2/" /etc/slurm/slurm.conf

sed -i "s/^\(SelectTypeParameters=.*\)/#\1/" /etc/slurm/slurm.conf

sed -i "s/^\(JobCompType=jobcomp\/none\)/#\1/" /etc/slurm/slurm.conf



cat >> /etc/slurm/slurm.conf << EOFslurm



PartitionName=xeon Nodes=${compute_prefix}[${first_node}-${last_node}] Default=YES MaxTime=24:00:00 State=UP

PartitionName=cluster Nodes=${compute_prefix}[${first_node}-${last_node}] Default=NO MaxTime=24:00:00 \

State=UP



EOFslurm



wwsh -y file import /etc/slurm/slurm.conf

sed -i "s/^files\(.*\)/files\1, slurm.conf, munge.key/" /etc/warewulf/defaults/provision.conf

syssystemctl enable slurmctld.service
```

### 2.2.3. Instalar el cliente Slurm en una imagen del nodo informático:

1. Agregue el cliente Slurm:

   ```
   dnf -y --installroot=$CHROOT install ohpc-slurm-client
   ```

2. En la imagen del nodo informático, cree este archivo: munge.key. Cuando el nodo está iniciado, la última copia de este archivo se usará.

   ```
   \cp -fa /etc/munge/munge.key $CHROOT/etc/munge/munge.key
   ```

3. Actualice la primera configuración Slurm en los nodos. La sincronización de Warewulf no sincronizar durante el parte temprano del proceso de iniciar de ordenador.

   ```
   \cp -fa /etc/slurm/slurm.conf $CHROOT/etc/slurm/
   ```

4. Habilite los servicios clientes Munge y los servicios clientes Slurm:

   ```
   systemctl --root=$CHROOT enable munge.service
   ```

5. Permita todo acceso SSH para usuarios en el grupo, sshallowlist, y el usuario “root”. Usuarios quien no están en este grupo, tienen que seguir las restricciones de SSH.

   a. Abre $CHROOT/etc/security/access.conf y agregue los siguientes líneas después de otras líneas. Mantenga el mismo orden de líneas.

      ```
      + : root : ALL
      ```

```
+ : sshallowlist : ALL
- : ALL : ALL
```

b. Guarde y cierre el archivo.

6. Usa el Gestionador de la Carga de Trabajo Slurm para permitir el control SSH. Permitir PAM en el entorno chroot estará limitado el acceso SSH a solo aquellos nodos donde el usuario tiene trabajos activos.

   a. Abra $CHROOT/etc/pam.d/sshd y agregue las lineas siguientes después de otras lineas:

   ```
   account sufficient pam_access.so
   account required pam_slurm.so
   ```

   b. Guarde y cierre el archivo.

   ```
   echo "account sufficient pam_access.so" >> $CHROOT/etc/pam.d/sshd
   ```

7. Actualice el imagen VNFS:

   ```
   wwvnfs --chroot $CHROOT –hybridize
   ```

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
echo "+ : root : ALL">>$CHROOT/etc/security/access.conf
echo "+ : sshallowlist : ALL">>$CHROOT/etc/security/access.conf echo "- : ALL : ALL">>$CHROOT/etc/security/access.conf
echo "account sufficient pam_access.so" >> $CHROOT/etc/pam.d/sshd
wwvnfs --chroot $CHROOT –hybridize
```

### 2.2.4. Opcional – Instalar el cliente Slurm en el primer nodo

Si quisiera usar el primer nodo en su clúster ejecutar las tareas que están programadas por Slurm, entonces use estas instrucciones para instalar el cliente Slurm:

1. Agregue el paquete del cliente Slurm en el primer nodo:

   ```
   dnf -y install ohpc-slurm-client
   ```

2. En la configuración Slurm, agregue el primer nodo y ajuste para que sea un nodo informático:

   a. Abre /etc/slurm/slurm.conf y encuentre la línea que contiene la especificación, NodeName. Si está usando la Plataforma de Procesadores Escalables Intel® Xeon® de 3a Generación Oro 6348 (en inglés: 3rd Generation Intel® Xeon® Scalable Gold 6348), copia esta línea para actualizar la NodeName:

   ```
   NodeName=frontend,c[01-04] Sockets=2 CoresPerSocket=28 ThreadsPerCore=2 State=UNKNOWN
   ```

   Encuentra la línea que contiene la especificación, PartitionName. Copie estas dos líneas para actualizarla:

   ```
   PartitionName=xeon Nodes=frontend,c[01-04] Default=YES MaxTime=24:00:00 State=UP
   PartitionName=cluster Nodes=frontend,c[01-04] Default=NO MaxTime=24:00:00 State=UP
   ```

b. Guarde y cierre el archivo.

3. Inicie el proceso de control Slurm:

```
systemctl restart slurmctld
```

4. Habilite e inicie el proceso control Slurm en el primer nodo:

```
systemctl enable --now slurmd
```

5. Actualice la configuración Slurm en los nodos informáticos:

```
\cp -fa /etc/slurm/slurm.conf $CHROOT/etc/slurm/
```

6. Actualice la imagen VNFS:

```
wwvnfs --chroot $CHROOT --hybridize
```

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
systemctl restart slurmctld

systemctl enable --now slurmd

\cp -fa /etc/slurm/slurm.conf $CHROOT/etc/slurm/

wwvnfs --chroot $CHROOT --hybridize
```

### 2.2.5. Completar la configuración de Slurm

1. La asignación de recursos en la versión Slurm 20.11.X ha cambiado. Varios tipos de trabajos MPI han sido afectados. Puede obtener más información: https://slurm.schedmd.com/archive/slurm-20.11.6/news.html

   En esta guía, usamos la versión Slurm más recente para hacer la asignación de recursos. En el primer nodo, tiene que ajustar el entorno variable, “SLURM?OVERLAP” para permitir a todos los usuarios.

   ```
   cat >> /etc/environment << EOFSLURM export SLURM_OVERLAP=1 EOFSLURM
   ```

2. Reinicie los servicios para controlar Munge y los servicios para controlar Slurm en el primer nodo:

   ```
   systemctl restart munge
   ```

3. Reinicie los nodos informáticos:

   ```
   pdsh -w c[01-XX] reboot
   ```

4. Asegúrese de que los nodos informáticos estén disponibles en Slurm. Póngalos en un estado de espera (en inglés: waiting or idle state) .

   ```
   scontrol reconfig
   ```

```
scontrol update NodeName=c[01-XX] State=Idle
```

5. Verifique el estado de Slurm. Todos nodos deben estar en estados de espera.

```
[root@frontend ~]# sinfo
PARTITION AVAIL      TIMELIMIT   NODES   STATE   NODELIST
xeon*            up 1-00:00:00     4      idle   c[01-04]
cluster          up 1-00:00:00     4      idle   c[01-04]
```

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
cat >> /etc/environment << EOFSLURM export SLURM_OVERLAP=1 EOFSLURM
systemctl restart munge
pdsh -w c[01-XX] reboot
scontrol reconfig
scontrol update NodeName=c[01-XX] State=Idle
[root@frontend ~]# sinfo
```

### 2.2.6. Verificar la configuración de Slurm

Verifique que los usuarios normales puedan ejecutar trabajos en el entorno de producción. Cree una cuenta de usuario normal nueva. Por ejemplo, este usuario no permite usar SSH fuera de un trabajo Slurm. Siga las siguientes instrucciones para crear una aplicación llamada “hola mundo de MPI”. Este se usa para verificar la configuración. Ejecute esta aplicación de forma interactiva. *Nota: se usa srun para ejecutar los trabajos en paralelo porque srun puede verificar que el trabajo nativo se está usando para iniciar esta tarea.*

1. Agregue el usuario, "test":

```
useradd -m test
passwd test
```

2. Sincronice los archivos con el base de datos Warewulf. *Nota: La sincronización de los nodos de informáticos puede tardar unos minutos.*

```
wwsh file resync
```

3. Cambie a otro usuario, "test":

```
su - test
```

4. Configure el entorno de la implementación MPI que se está usando, por ejemplo, use el entorno Intel® MPI de oneAPI:

```
module load oneapi
```

5. Compile el programa, “hola al mundo de MPI”:

```
mpicc -o test /opt/intel/oneapi/mpi/latest/test/test.c
```

6. Inicie el trabajo Slurm en todos nodos. Este trabajo debe producir resultados como esos:

```
[test@frontend ~]# srun --mpi=pmi2 --partition=xeon -N 4 -n 4 /home/test/test

 Hello world: rank 0 of 4 running on c01.cluster
```

7. Cuando este trabajo se ejecute con éxito, logout el usuario, "test". Continúe con la configuración del clúster.

```
exit
```

## 2.3. Instalar las herramientas de análisis genómico

Las herramientas incluyen el kit de herramientas de análisis genómico (GATK por su sigla en inglés) y el sistema Cromwell de gestión de flujo de trabajo.

### 2.3.1. Verificar la configuración del entorno del clúster genómico

Primero, verifique que la configuración del clúster que usa para ejecutar el análisis genómico sea correcta.

1. Los ajustes de carga de trabajo pueden abrir 500.000 archivos en cada nodo. Ajuste este limite:

```
pdsh -w frontend,c0[1-4] "su - cromwell -c 'ulimit -n'"
```

2. Verifique que el */genomics_local* es un sistema de archivos montados con los permisos de archivos correctos y los derechos de propiedad correctos. El mandato pdsh debe producir resultados similares con estos, excepto el orden de los nodos de computación pueda estar diferentes.

```
[root@frontend ~]# pdsh -w c0[1-4] "stat -c '%A %U:%G %m' /genomics_local"

 c01: drwxr-xr-x cromwell:cromwell /genomics_local

 c03: drwxr-xr-x cromwell:cromwell /genomics_local

 c04: drwxr-xr-x cromwell:cromwell /genomics_local

 c02: drwxr-xr-x cromwell:cromwell /genomics_local
```

### 2.3.2. Configurar la base de datos, MariaDB

Cromwell usa MariaDB para guardar información de los trabajos. Porque Warewulf ya usa MariaDB, la base de datos debería configurar. Las instrucciones de configuración están en la guía, *HPC Cluster Tuning on 3rd Generation Intel® Xeon® Scalable Processors*. Si no están, entonces use las siguientes instrucciones para configurarla. Necesita la contraseña del administrador.

1. Permita el servicio de la base de datos iniciar automáticamente durante el inicio del sistema (boot up):

```
systemctl enable mariadb.service
```

2. Permita al servicio de la web iniciar automáticamente durante el inicio del sistema:

```
systemctl enable httpd.service
```

3. Reinicie el servicio de la base de datos:

```
systemctl restart mariadb
```

4. Reinicie el servicio de la web:

```
systemctl restart httpd
```

5. Actualice la contraseña de la base de datos Warewulf:
   a. Edite el archivo: /etc/warewulf/database-root.conf
   b. Cambie la contraseña "changeme" a otra contraseña. Use su directiva de contraseñas para ayudar a elegir una contraseña segura.
   ```
   database password = <new_password>
   ```
   c. Le recomendamos que la contraseña de la base de datos es diferente que la contraseña del usuario root (superuser).
6. Guarde el archivo.
7. MariaDB tiene una herramienta que permite al administrador configurar el acceso seguro de la base de datos. Esta herramienta se debe usar para ajustar la contraseña del administrador MariaDB. Le recomendamos que la contraseña del administrador MariaDB sea diferente que la contraseña del usuario root del sistema y también sea diferente que la contraseña del administrador de la base de datos, Warewulf. Use su directiva de contraseñas para ayudar elegir una contraseña segura. Para ajustar la contraseña, ejecute este comando luego siga las instrucciones. Le recomendamos encarecidamente que use las repuestas predeterminadas para contestar todas las preguntas.

```
/usr/bin/mysql_secure_installation
```

8. Inicialice la base de datos Warewulf. Introduzca la contraseña de esta base de dato cuando se le solicite.

```
wwinit DATASTORE
```

9. Use la cuente del administrador para iniciar una sesión al servidor MariaDB. Introduzca la contraseña root cuando se le solicite.

```
mysql -h localhost -u root -p
```

*Nota: Si no puede iniciar sesión, puede leer la sección en esta guía, Resolviendo problemas con el software genómico.*

10. Cree una nueva base de datos llamada, “cromwell”:

```
MariaDB [(none)]> CREATE DATABASE cromwell;
```

11. Cree una nueva cuenta de usuario. Use su directiva de contraseñas para ayudar a elegir una contraseña segura. En este ejemplo, la cuenta de usuario y la contraseña son ambos “cromwell”.

```
MariaDB [(none)]> CREATE USER 'cromwell'@'localhost' IDENTIFIED BY 'cromwell';
```

12. Permita al usuario acceder a la base de datos. Sustituye “cromwell” en 'cromwell'@'localhost' con su contraseña .

```
MariaDB [(none)]> GRANT ALL PRIVILEGES ON `cromwell`.* TO 'cromwell'@'localhost';
```

13. Reinicie la base de datos:

```
 MariaDB [(none)]> FLUSH PRIVILEGES;
```

14. Salga de la base de datos:

```
MariaDB [(none)]> exit
```

### 2.3.3. Instalar el sistema de gestión de flujo de trabajo del Cromwell

Instalar el sistema de gestión de flujo de trabajo del Cromwell. Configurarlo para utilizar el disco duro que está conectado físicamente a los nodos informáticos (en inglés: local scratch device) .

1. Para instalar el Cromwell, use la herramienta de compilación "sbt". Primero, instale sbt:
   a. Agregue sbt al repositorio en línea:

      ```
      dnf config-manager --add-repo https://www.scala-sbt.org/sbt-rpm.repo
      ```

   b. Instale la herramienta de compilación sbt:

      ```
      dnf -y install sbt
      ```

2. Descargue el repositorio git :
   a. Cree una carpeta para instalar Cromwell:

      ```
      mkdir -p ${GENOMICS_PATH}/cromwell
      ```

   b. Cambie la propiedad de la carpeta al usuario, *cromwell.* También, agregue el grupo, cromwell, a la propiedad de la carpeta. Cambie los derechos de propiedad para permitir el acceso.

      ```
      chmod 775 -R ${GENOMICS_PATH}/Cromwell
      chown -R cromwell:cromwell ${GENOMICS_PATH}/Cromwell
      ```

   c. Inicie sesión usando la cuenta, *cromwell*:

      ```
      su - cromwell
      ```

   d. Copie el repositorio git Cromwell:

      ```
      cd ${GENOMICS_PATH}/cromwell
      git clone https://github.com/broadinstitute/cromwell.git
      ```

   e. Esta guía se ha verificado usando la versión Cromwell 52. Le recomendamos que use esta versión.

      ```
      cd cromwell
      git checkout 52
      ```

3. Configure Cromwell para usar el disco duro NVMe local para espacio provisional:
   a. Edite este archivo:

      ```
      backend/src/main/scala/cromwell/backend/RuntimeEnvironment.scala
      ```

   b. Debe cambiar la línea 3 por una acotación:

      ```
      //import java.util.UUID
      ```

   c. Actualice las líneas de 23 a 27:

      ```
      val tempPath: String = {
      ```

```
val uuid = UUID.randomUUID().toString
val hash = uuid.substring(0, uuid.indexOf('-'))
callRoot.resolve(s"tmp.$hash").pathAsString
}
```

d. Agregue este texto a la línea 23:

```
def tempPath: String = "/genomics_local"
```

e. Guarde el archivo y salga

f. Abra este archivo:

```
backend/src/main/scala/cromwell/backend/standard/ StandardAsyncExecutionActor.scala
```

g. Vaya la línea numero 380 para encontrar este texto:

```
|export _JAVA_OPTIONS=-Djava.io.tmpdir="$$tmpDir"
|export TMPDIR="$$tmpDir"
```

h. Elimine las dos líneas y agregue este texto:

```
|mkdir -p $$tmpDir/tmp.$$$$
|export _JAVA_OPTIONS=-Djava.io.tmpdir="$$tmpDir/tmp.$$$$"
|export TMPDIR="$$tmpDir/tmp.$$$$
```

i. Guarde el archivo y salga.

4. Compile Cromwell con los parches:

```
sbt clean
```

5. Después de compilar, mueva el archivo nuevo a ${GENOMICS_PATH}/cromwell

```
cp server/target/scala-2.12/cromwell-52-*-SNAP.jar \
${GENOMICS_PATH}/cromwell/cromwell-52-fix.jar
```

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
sed -i "s/^\(import\ java.util.UUID\)/\/\/\1/" \
backend/src/main/scala/cromwell/backend/RuntimeEnvironment.scala
sed -i '23,27d' \ backend/src/main/scala/cromwell/backend/RuntimeEnvironment.scala

sed -i '23i \ \ \ \ \ \ def tempPath: String = \"/genomics_local\"' \
backend/src/main/scala/cromwell/backend/RuntimeEnvironment.scala

sed 's/\(\s*|export _JAVA_OPTIONS.*\)\"/\ \ \ \ \ \ \ \ |mkdir -p \$\$tmpDir\/tmp\.\$\$\$\$\n\1\"/' \
backend/src/main/scala/cromwell/backend/standard/StandardAsyncExecutionActor.scala
```

```
sed 's/\(\s*|export _JAVA_OPTIONS.*tmpDir\)\"/\1\/tmp\.\$\$\$\$\$\"/' \
backend/src/main/scala/cromwell/backend/standard/StandardAsyncExecutionActor.scala


sed 's/\(\s*|export TMPDIR=.*tmpDir\)\"/\1\/tmp\.\$\$\$\$\$\"/' \
backend/src/main/scala/cromwell/backend/standard/StandardAsyncExecutionActor.scala
```

*Edit backend/src/main/scala/cromwell/backend/standard/ StandardAsyncExecutionActor.scala*

*Replace line 380-381 with:*

```
|mkdir -p $$tmpDir/tmp.$$$$

|export _JAVA_OPTIONS=-Djava.io.tmpdir="$$tmpDir/tmp.$$$$"

|export TMPDIR="$$tmpDir/tmp.$$$$


sbt clean

cp server/target/scala-2.12/cromwell-52-*-SNAP.jar ${GENOMICS_PATH}/cromwell/cromwell-52-fix.jar
```

### 2.3.4. Configurar el entorno Cromwell

Use la configuración predeterminada para iniciar, entonces puede hacer cambios si es necesario.

- Configure la base de datos de MariaDB con la misma base de datos que usa Cromwell.
- Agregue los programas de Slurm para que Cromwell pueda usar Slurm para crear un horario de trabajos

**Instrucciones paso a paso**

1. Primero, descargue el archivo de configuración predeterminada, *reference.conf*

   ```
   wget https://raw.githubusercontent.com/broadinstitute/cromwell/52_hotfix/core/\
   ```

2. Configure la base de datos de MariaDB con la misma base de datos que usa Cromwell:

   a. Abra esto archivo: *${GENOMICS_PATH}/cromwell/reference.conf*

   b. Agregue las siguientes líneas después de otras líneas:

   ```
   database {
   profile = "slick.jdbc.MySQLProfile$"db {
   driver = "com.mysql.cj.jdbc.Driver"
   url = "jdbc:mysql://localhost/cromwell?rewriteBatchedStatements=true&serverTimezone=UTC"
   user = "cromwell"
   ```

```
password = "cromwell" connectionTimeout = 5000
}
}
```

c. Guarde el archivo y salga.

3. Cambie la configuración de Cromwell para usar los programas Slurm

a. Abra este archivo: *${GENOMICS_PATH}/cromwell/reference.conf*

b. Vaya a la línea 479:

```
default = "Local"
```

c. Actualice esta línea y cambie `"Local"` a `"SLURM"`:

```
default = "SLURM"
```

d. Elimine las siguientes cinco líneas (las líneas con números de 480 a 484)

```
providers {
Local {
actor-factory = "cromwell.backend.impl.sfs.config.ConfigBackendLifecycleActorFactory"
config {
include  required(classpath("reference_local_provider_config.inc.conf"))
```

e. Encuentre la línea 479:

```
default = "SLURM"
```

Agregue el siguiente texto después de la línea 479.

*Nota: Verifique que las líneas que tiene saltos de línea "CRLF" (en inglés: line-breaks) en esta guía, son también líneas individuales in reference.conf*

```
providers { SLURM {
# Modifying temp directory to write to local disk temporary-directory = "$(/genomics_local/)"
actor-factory = "cromwell.backend.impl.sfs.config.ConfigBackendLifecycleActorFactory" config {
root = "cromwell-slurm-exec" runtime-attributes = """
Int runtime_minutes = 600 Int cpu = 2
Int memory_mb = 1024 String queue = "all"
String? docker """
submit = """
sbatch -J ${job_name} -D ${cwd} -o ${out} -e ${err} -t ${runtime_minutes} -p ${queue} ${"-c " +
cpu} --mem ${memory_mb} --wrap "/bin/bash ${script}"
"""
kill = "scancel ${job_id}"
check-alive = "squeue -j ${job_id}"
job-id-regex = "Submitted batch job (\\d+).*"
}
}
SLURM-BWA {
temporary-directory = "$(/genomics_local/)"
actor-factory = "cromwell.backend.impl.sfs.config.ConfigBackendLifecycleActorFactory" config {
root = "cromwell-slurm-exec" runtime-attributes = """
Int runtime_minutes = 600 Int cpu = 2
Int memory_mb = 1024 String queue = "bwa"
String? docker """
submit = """
```

```
    sbatch -J ${job_name} -D ${cwd} -o ${out} -e ${err} -t ${runtime_minutes} -p ${queue} ${"-c " +
    cpu} --mem ${memory_mb} --wrap "/bin/bash ${script}"
    """
    kill = "scancel ${job_id}"
    check-alive = "squeue -j ${job_id}"
    job-id-regex = "Submitted batch job (\\d+).*"
    }
    }
    SLURM-HAPLO {
    temporary-directory = "$(/genomics_local/)"
    actor-factory = "cromwell.backend.impl.sfs.config.ConfigBackendLifecycleActorFactory" config {
    root = "cromwell-slurm-exec" runtime-attributes = """
    Int runtime_minutes = 600 Int cpu = 2
    Int memory_mb = 1024 String queue = "haplo"
    String? docker """
    submit = """
    sbatch -J ${job_name} -D ${cwd} -o ${out} -e ${err} -t ${runtime_minutes} -p ${queue} ${"-c " +
    cpu} --mem ${memory_mb} --wrap "/bin/bash ${script}"
    """
    kill = "scancel ${job_id}"
    check-alive = "squeue -j ${job_id}"
    job-id-regex = "Submitted batch job (\\d+).*"
```

f. Guarde el archivo y salga.

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
sed -i '$ a database {' reference.conf

sed -i '$ a \ \ profile = \"slick.jdbc.MySQLProfile$\"' reference.conf sed -i '$ a \ \ db {' reference.conf

sed -i '$ a \ \ \ \ driver = \"com.mysql.cj.jdbc.Driver\"' reference.conf

sed -i '$ a \ \ \ \ url = \"jdbc:mysql:\/\/localhost\/cromwell\?rewriteBatched\ Statements=true&serverTimezone=UTC\"' reference.conf

sed -i '$ a \ \ \ \ user = \"cromwell\"' reference.conf

sed -i '$ a \ \ \ \ password = \"cromwell\"' reference.conf sed -i '$ a \ \ \ \ connectionTimeout = 5000' reference.conf sed -i '$ a \ \ }' reference.conf

sed -i '$ a }' reference.conf

sed -i '479,484d' reference.conf

sed -i "479i \ \ \ \ \ \ \ \ job-id-regex\ =\ \"Submitted\ batch\ job\ (\\\\\\\\d+).*\"" \ reference.conf

sed -i "479i \ \ \ \ \ \ \ \ check-alive\ =\ \"squeue\ -j\ \${job_id}\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ kill\ =\ \"scancel\ \${job_id}\"" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ \"\"\"" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ sbatch\ -J\ \${job_name}\ -D\ \${cwd}\ -o\ \${out}\ -e\ \${err}\ \

-t\ \${runtime_minutes}\ -p\ \${queue}\ \${\"-c\ \"\ +\ cpu}\ --mem\ \${memory_mb}\ \

--wrap\ \"\/bin\/bash\ \${script}\"" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ submit\ =\ \"\"\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ \"\"\"" reference.conf
```

```
sed -i "479i \ \ \ \ \ \ \ \ String?\ docker" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ String\ queue\ =\ \"haplo\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ Int\ memory_mb\ =\ 1024" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ Int\ cpu\ =\ 2" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ Int\ runtime_minutes\ =\ 600" reference.conf sed -i "479i \ \ \ \ \ \ \ \ runtime-attributes\ =\ \"\"\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ root\ =\ \"cromwell-slurm-exec\"" reference.conf sed -i "479i \ \ \ \ \ \ config\ {" reference.conf



sed -i "479i \ \ \ \ \ \ actor-factory\ =\ \"cromwell.backend.impl.sfs.config.\ ConfigBackendLifecycleActorFactory\"" reference.conf

sed -i "479i \ \ \ \ \ \ temporary-directory\ =\ \"\$(\/genomics_local\/)\"" reference.conf sed -i "479i \ \ \ \ SLURM-HAPLO\ {" reference.conf

sed -i "479i \ \ \ \ }\ \ " reference.conf sed -i "479i \ \ \ \ \ \ }" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ job-id-regex\ =\ \"Submitted\ batch\ job\ (\\\\\\\\d+).*\"" \ reference.conf

sed -i "479i \ \ \ \ \ \ \ \ check-alive\ =\ \"squeue\ -j\ \${job_id}\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ kill\ =\ \"scancel\ \${job_id}\"" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ \"\"\"" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ sbatch\ -J\ \${job_name}\ -D\ \${cwd}\ -o\ \${out}\ -e\ \${err}\ \

-t\ \${runtime_minutes}\ -p\ \${queue}\ \${\"-c\ \"\ +\ cpu}\ --mem\ \${memory_mb}\ \

--wrap\ \"\/bin\/bash\ \${script}\"" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ submit\ =\ \"\"\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ \"\"\"" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ String?\ docker" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ String\ queue\ =\ \"bwa\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ Int\ memory_mb\ =\ 1024" reference.conf sed -i "479i \ \ \ \ \ \ \ \ Int\ cpu\ =\ 2" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ Int\ runtime_minutes\ =\ 600" reference.conf sed -i "479i \ \ \ \ \ \ \ \ runtime-attributes\ =\ \"\"\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ root\ =\ \"cromwell-slurm-exec\"" reference.conf sed -i "479i \ \ \ \ \ \ config\ {" reference.conf

sed -i "479i \ \ \ \ \ \ actor-factory\ =\ \"cromwell.backend.impl.sfs.config.\ ConfigBackendLifecycleActorFactory\"" reference.conf

sed -i "479i \ \ \ \ \ \ temporary-directory\ =\ \"\$(\/genomics_local\/)\"" reference.conf sed -i "479i \ \ \ \ SLURM-BWA\ {" reference.conf

sed -i "479i \ \ \ \ }" reference.conf

sed -i "479i \ \ \ \ \ \ }\ \ " reference.conf

sed -i "479i \ \ \ \ \ \ \ \ job-id-regex\ =\ \"Submitted\ batch\ job\ (\\\\\\\\d+).*\"" \ reference.conf

sed -i "479i \ \ \ \ \ \ \ \ check-alive\ =\ \"squeue\ -j\ \${job_id}\"" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ kill\ =\ \"scancel\ \${job_id}\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ \"\"\"" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ sbatch\ -J\ \${job_name}\ -D\ \${cwd}\ -o\ \${out}\ -e\ \${err}\ \

-t\ \${runtime_minutes}\ -p\ \${queue}\ \${\"-c\ \"\ +\ cpu}\ --mem\ \${memory_mb}\ \

--wrap\ \"\/bin\/bash\ \${script}\"" reference.conf
```

```
sed -i "479i \ \ \ \ \ \ \ \ submit\ =\ \"\"\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ \"\"\"" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ String?\ docker" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ String\ queue\ =\ \"all\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ Int\ memory_mb\ =\ 1024" reference.conf sed -i "479i \ \ \ \ \ \ \ \ Int\ cpu\ =\ 2" reference.conf

sed -i "479i \ \ \ \ \ \ \ \ Int\ runtime_minutes\ =\ 600" reference.conf sed -i "479i \ \ \ \ \ \ \ \ runtime-attributes\ =\ \"\"\"" reference.conf sed -i "479i \ \ \ \ \ \ \ \ root\ =\ \"cromwell-slurm-exec\"" reference.conf sed -i "479i \ \ \ \ \ \ config\ {" reference.conf

sed -i "479i \ \ \ \ \ \ actor-factory\ =\ \"cromwell.backend.impl.sfs.config.\ ConfigBackendLifecycleActorFactory\"" reference.conf

sed -i "479i \ \ \ \ \ \ temporary-directory\ =\ \"\$(\/genomics_local\/)\"" reference.conf

sed -i "479i \ \ \ \ \ \ #\ Modifying\ temp\ directory\ to\ write\ to\ local\ disk" reference.conf sed -i "479i \ \ \ \ SLURM\ {" reference.conf

sed -i "479i \ \ providers\ {" reference.conf

sed -i "479i \ \ default\ =\ \"SLURM\"" reference.conf
```

### 2.3.5. Verificar la configuración de Cromwell

Verifique que la instalación Cromwell este trabajando correctamente.

1. Cambie a otro usuario, "cromwell":

   ```
   su - cromwell
   ```

2. Cambie a la carpeta, *cromwell*

   ```
   cd ${GENOMICS_PATH}/cromwell
   ```

3. Inicie el servidor Cromwell como un proceso separado que está ejecutándose en el fondo. El servidor Cromwell pueda obtener el acceso a través de un puerto 8000. El archivo, cromwell.log, estará contenido de los mensajes que el servidor Cromwell reportará durante la ejecución.

   ```
   nohup java -jar -Dconfig.file=reference.conf cromwell-52-fix.jar server 2>&1 >>cromwell.log &
   ```

4. Use el mandato “ps” para ver una lista de los procesos para confirmar que el servidor Cromwell está ejecutando

   *Nota: El mandato “ps” reporta el PID y otra información. En este ejemplo, <...> es un sustitución de esa información.*

   ```
   [cromwell@frontend cromwell]$ ps aux | grep cromwell | grep server
   ```

   ```
   cromwell <PID> <…> java -jar -Dconfig.file=reference.conf cromwell-52-fix.jar server
   ```

   *Nota: Para dejar el servidor Cromwell, use el mandato: "kill -9 <PID>". <PID> es el número del proceso que está ejecutando el servidor Cromwell.*

5. Cuando el proceso del servidor Cromwell está ejecutándose, entonces ejecute un ejemplo de carga de trabajo.

Esta carga de trabajo se ha diseñado para usar el Lenguaje de Descripción del Flujo de Trabajo (WDL por su sigla en inglés).

a. Vaya a la carpeta proyecto:

```
cd ${GENOMICS_PATH}/cromwell/
```

b. Abra un archivo nuevo que se llama, *HelloWorld.wdl.* Agregue el siguiente texto:

```
task hello {
String name command {
echo 'Hello ${name}!'
}
output {
File response = stdout()
}
runtime {
memory: "2MB" disk: "2MB"
```

c. Guarde el archivo y salga.

d. Abra un archivo nuevo que se llama: *HelloWorld.json*

e. Agregue el siguiente texto:

```
{"helloWorld.hello.name": "World"}
```

f. Guarde el archivo y salga.

g. Envié una carga de trabajo al servidor Cromwell. Esta carga de trabajo contiene los archivos WDL y los archivos JSON, que se acaban de crear:

```
curl -v http://127.0.0.1:8000/api/workflows/v1 -F \
workflowSource=@${GENOMICS_PATH}/cromwell/HelloWorld.wdl -F \
workflowInputs=@${GENOMICS_PATH}/cromwell/HelloWorld.json
```

h. Después de ejecutar una carga de trabajo, verá una identificación de trabajo en los resultados. Para verificar el éxito de la carga de trabajo, use esta identificación de trabajo en este mandato:

```
curl -v http://127.0.0.1:8000/api/workflows/v1/<id>/status

#After a few minutes, the above command should return a "succeeded" message.
```

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
su – Cromwell

cd ${GENOMICS_PATH}/cromwell

nohup java -jar -Dconfig.file=reference.conf cromwell-52-fix.jar server 2>&1 >>cromwell.log &
```

```
[cromwell@frontend cromwell]$ ps aux | grep cromwell | grep server

cromwell <PID> <…> java -jar -Dconfig.file=reference.conf cromwell-52-fix.jar server

cd ${GENOMICS_PATH}/cromwell/


cat >> HelloWorld.wdl << EOFwdl task hello {

String name command {

echo 'Hello ${name}!'

}

output {

File response = stdout()

}

runtime {

memory: "2MB" disk: "2MB"

}

}

workflow helloWorld { call hello

}

EOFwdl


echo {\"helloWorld.hello.name\":\ "World\"}" >> HelloWorld.json


curl -v http://127.0.0.1:8000/api/workflows/v1 -F \ workflowSource=@${GENOMICS_PATH}/cromwell/HelloWorld.wdl -F \
workflowInputs=@${GENOMICS_PATH}/cromwell/HelloWorld.json


curl -v http://127.0.0.1:8000/api/workflows/v1/<id>/status


#After a few minutes, the above command should return a “succeeded” message.
```

### 2.3.6. Instalar el paquete software Burrows-Wheeler Aligner (BWA)

Burrows-Wheeler Aligner (BWA) 0.7.17 se usa para comparar las secuencias del ADN con un genoma de referencia como el genoma de referencia humano. Antes de que pueda usarlo, tiene que compilarlo.

1. Cambie a otro usuario, "cromwell":

   ```
   su - cromwell
   ```

2. Cree una subcarpeta, *tools,* en la carpeta: GENOMICS_PATH. Vaya a esta subcarpeta:

   ```
   mkdir ${GENOMICS_PATH}/tools
   cd tools
   ```

3. Descargue la versión de BWA que se recomienda:

   ```
   wget https://github.com/lh3/bwa/releases/download/v0.7.17/bwa-0.7.17.tar.bz2
   ```

4. Desempaqueta el software de su formato de distribución (normalmente en un “tarball”):

   ```
   tar -xjf bwa-0.7.17.tar.bz2
   ```

5. Compile BWA:

   ```
   cd bwa-0.7.17
   ```

6. Haga un symlink:

   ```
   cd ${GENOMICS_PATH}/tools
   ```

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
su - cromwell

mkdir ${GENOMICS_PATH}/tools

wget https://github.com/lh3/bwa/releases/download/v0.7.17/bwa-0.7.17.tar.bz2

tar -xjf bwa-0.7.17.tar.bz2
```

```
cd bwa-0.7.17



cd ${GENOMICS_PATH}/tools
```

### 2.3.7. Instalar el kit de herramientas de análisis genómico (GATK)

El kit de herramientas de análisis genómico (GATK por su sigla en inglés) es usando para analizar datos de secuenciación genética y para el descubrimiento de variantes. Es la norma para identificación de SNPs e "indels" en el ADN de la línea germinal y datos ARNseq. Incluye utilidades para hacer tareas como procesar y controlar la calidad de datos de secuencia de alto rendimiento.

Estas herramientas se pueden usar individualmente o juntos en las cargas de trabajo. El Instituto Broad provee las cargas de trabajo y se llaman "Mejores Prácticas GATK". Hay diferentes cargas de trabajo dependiendo de sus necesidades. Las mejoras de rendimiento del Intel® Genomics Kernel Library with AVX-512 están incluidas en GATK.

1. Vaya a su carpeta proyecto:

   ```
   cd ${GENOMICS_PATH}/tools
   ```

2. Descargue la última versión de GATK:

   ```
   wget \
   https://github.com/broadinstitute/gatk/releases/download/4.1.9.0/gatk-4.1.9.0.zip
   ```

3. Desempaquetar el archivo zip:

   ```
   unzip gatk-4.1.9.0.zip
   ```

4. Agregue la ruta del archivo al entorno del usuario predeterminado.

   ```
   echo "export PATH=\${GENOMICS_PATH}/tools/gatk-4.1.9.0:\$PATH" >> /etc/bashrc
   ```

5. Haga el /tools symlink:

   ```
   cd ${GENOMICS_PATH}/tools
   ln -s gatk-4.1.9.0 gatk
   ln -s gatk-4.1.9.0/gatk-package-4.1.9.0-local.jar gatk-jar
   ```

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
cd ${GENOMICS_PATH}/tools
wget \
https://github.com/broadinstitute/gatk/releases/download/4.1.9.0/gatk-4.1.9.0.zip
unzip gatk-4.1.9.0.zip
echo "export PATH=\${GENOMICS_PATH}/tools/gatk-4.1.9.0:\$PATH" >> /etc/bashrc
```

```
cd ${GENOMICS_PATH}/tools
ln -s gatk-4.1.9.0 gatk
ln -s gatk-4.1.9.0/gatk-package-4.1.9.0-local.jar gatk-jar
```

### 2.3.8. Instalar Picard

1. Vaya a su carpeta proyecto:

   ```
   cd ${GENOMICS_PATH}/tools
   ```

2. Descargue la versión recomendada de Picard:

   ```
   wget https://github.com/broadinstitute/picard/releases/download/2.21.1/picard.jar
   ```

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
cd ${GENOMICS_PATH}/tools
wget https://github.com/broadinstitute/picard/releases/download/2.21.1/picard.jar
```

### 2.3.9. Instalar Samtools

1. Cambie a la carpeta, *tools*

   ```
   cd ${GENOMICS_PATH}/tools
   ```

2. Descargue la versión recomendada de SAMtools:

   ```
   wget https://github.com/samtools/samtools/releases/download/1.9/samtools-1.9.tar.bz2
   ```

3. Desempaquetar el archivo tarball:

   ```
   tar -xjf samtools-1.9.tar.bz2
   ```

4. Compile y instale Samtools:

   ```
   cd samtools-1.9
   ./configure -prefix=${GENOMICS_PATH}/tools
   make
   ```

5. Haga un symlink:

   ```
   ${GENOMICS_PATH}/tools
   ln -s samtools-1.9 samtools
   ```

**Un resumen de los comandos**

Todos los comandos en esta sección están aquí:

```
cd ${GENOMICS_PATH}/tools
wget https://github.com/samtools/samtools/releases/download/1.9/samtools-1.9.tar.bz2
tar -xjf samtools-1.9.tar.bz2
cd samtools-1.9
./configure -prefix=${GENOMICS_PATH}/tools
make
${GENOMICS_PATH}/tools
ln -s samtools-1.9 samtools
```

### 2.3.10. Instalar VerifyBamID2

Tiene que compilar el proyecto del código original. Puede obtener más información: https://github.com/Griffan/VerifyBamID

1. Vaya a su carpeta proyecto:

   ```
   cd ${GENOMICS_PATH}/tools
   ```

2. Cree un archivo, *build_verify.sh*

   ```
   #https://github.com/broadinstitute/warp/tree/cec97750e3819fd88ba382534aaede8e05ec52df/dockers/broad/ VerifyBamId
   cd $GENOMICS_PATH/tools

   VERIFY_BAM_ID_COMMIT="c1cba76e979904eb69c31520a0d7f5be63c72253" GIT_HASH=$VERIFY_BAM_ID_COMMIT
   HTS_INCLUDE_DIRS=$GENOMICS_PATH/tools/samtools-1.9/htslib-1.9/
   HTS_LIBRARIES=$GENOMICS_PATH/tools/samtools-1.9/htslib-1.9/libhts.a

   wget -nc https://github.com/Griffan/VerifyBamID/archive/$GIT_HASH.zip && \ unzip -o
   $GIT_HASH.zip && \
   cd VerifyBamID-$GIT_HASH && \ mkdir build && \
   cd build && \
   echo cmake -DHTS_INCLUDE_DIRS=$HTS_INCLUDE_DIRS -DHTS_LIBRARIES=$HTS_LIBRARIES .. && \
   CC=$(which gcc) CXX=$(which g++) \
   cmake -DHTS_INCLUDE_DIRS=$HTS_INCLUDE_DIRS -DHTS_LIBRARIES=$HTS_LIBRARIES .. && \
   make && \
   make test && \ cd ../../
   mv $GENOMICS_PATH/tools/VerifyBamID-$GIT_HASH $GENOMICS_PATH/tools/VerifyBamID && \ rm -rf
   $GENOMICS_PATH/tools/$GIT_HASH.zip $GENOMICS_PATH/tools/VerifyBamID-$GIT_HASH
   ```

3. Instale VerifyBamID2:

   ```
   sh ./build_verify.sh
   ```

## 3. Verificar la configuración

La prueba de alto rendimiento ejecuta 20.000 veces (en el inglés: 20K Throughput Run) para hacer una evaluación

comparativa. Se usa para verificar que las funcionas más importantes puedan funcionar bien. Se usa para ejecutar las Mejores Prácticas de la Cargas de Trabajo Broad con una base de datos cortos. Mientras una secuencia de genoma completo individual puede ejecutar por horas, esta prueba lo puede completar en 30 o 40 minutos.

1. Inicie sesión usando la cuenta, cromwell:

   ```
   su - cromwell
   ```

2. Vaya a la carpeta proyecto, cromwell

   ```
   cd ${GENOMICS_PATH}/cromwell
   ```

3. Use la cuenta de un usuario regular y copie el repositorio de carga de trabajo. Configure los ajustes de Git proxy si necesario.

   ```
   git clone https://github.com/Intel-HLS/BIGstack.git
   ```

4. Repase los ajustes y verifique que las rutas de carpetas DATA y TOOL son correctos. DATA debe estar en un sistema de carpetas que todos los nodos pueden tener el acceso. Este sistema de carpetas necesita por lo menos 2TB de espacio libre.

   ```
   vim configure
   ```

5. Configure la prueba basada en los detalles en el archivo "configure":

   ```
   ./step01_Configure_20k_Throughput-run.sh
   ```

6. Descargue los datos que se ha proveido en el archivo. Configure a la DATA PATH:

   ```
   ./step02_Download_20k_Data_Throughput-run.sh
   ```

7. Ejecute el rendimiento de referencia:

   ```
   ./step03_Cromwell_Run_20k_Throughput-run.sh
   ```

8. Verifique el estado de la carga de trabajo. Hay tres estados: ejecutando, fallido o exitoso:

   ```
   ./step04_Cromwell_Monitor_Single_Sample_20k_Workflow.sh
   ```

9. Para verificar los últimos resultados de esta prueba, ejecute:

   *step05_Single_Sample_20k_Workflow_Output.sh*

   ```
   ./step05_Single_Sample_20k_Workflow_Output.sh
   ```

   ```
   Total Elapsed Time for 64 workflows: 'X' minutes: 'Y' seconds
   Average Elapsed Time for Mark Duplicates: 'X.YZ' minutes
   ```

# 4. Resolviendo problemas con el software genómicas

La siguiente sección contiene los pasos que pueden ayudar a resolver problemas durante la instalación del software genómicas.

## 4.1. Resolviendo problemas iniciando una sesión con MariaDB

1. Pare MariaDB:

   ```
   systemctl stop mariadb
   ```

2. Inicie una sesión segura:

```
mysqld_safe --skip-grant-tables --skip-networking & mysql -u root
```

3. Cambie la contraseña del usuario, root:

```
MariaDB [(none)]> FLUSH PRIVILEGES;

MariaDB [(none)]> SET PASSWORD FOR 'root'@'localhost' = PASSWORD('password'); MariaDB
[(none)]> EXIT;
```

4. Reinicie MariaDB:

```
kill `cat /var/run/mariadb/mariadb.pid` systemctl start mariadb
```

5. Inicie una sesión en el servidor MariaDB usando el usuario, administrador. Introduzca la contraseña cuando se le solicite.

```
mysql -h localhost -u root -p
```

## 5. Instalar componentes opcionales

El Intel® System Configuration Utility (SYSCFG) es una utilidad de línea de comandos para cargar y recargar el sistema BIOS y para gestionar los ajustes del firmware.

## 6. Conclusión

Esta guía incluye las recomendaciones para configurar el Intel® Select Solutions for Genomics Analytics que se puede aumentar el rendimiento en muchas situaciones. En la guía llamada, HPC Cluster Tuning on 3rd Generation Intel® Xeon® Scalable Processors, le recomienda que aumente el rendimiento usando las configuraciones del hardware. En esta guía, le recomendamos que aumente el rendimiento usando estas configuraciones del software.

**Notices & Disclaimers**

Intel technologies may require enabled hardware, software or service activation.

No product or component can be absolutely secure.

Your costs and results may vary.

Code names are used by Intel to identify products, technologies, or services that are in development and not publicly available. These are not "commercial" names and not intended to function as trademarks